



BASES DE DONNÉES

CM :

*Langage de Manipulation de Données (LMD) -
SIMPLE*

Vincent COUTURIER

Maitre de conférences – Université Savoie Mont-Blanc

vincent.couturier@univ-smb.fr

Préambule

- Schéma de la base de données EMPLOYES :

DEPT(DEPTNO, DNAME, LOC)

EMP(EMPNO, ENAME, JOB, #MGR, HIREDATE,
SAL, COMM, ADR, #DEPTNO)

SELECT (RAPPELS)

Clauses du SELECT

SELECT	<liste des attributs projetés>
FROM	<liste des relations touchées par la question>
[WHERE	<liste des critères de restriction>
GROUP BY	<liste des attributs d'agrégation>
HAVING	<liste des critères de restriction sur les agrégats>
ORDER BY	<liste des attributs de tri du résultat>]

Il est obligatoire de respecter cet ordre !

[] = clauses optionnelles

Rappels

- Quelques rappels :

- IS NULL (= valeur non renseignée)

```
SELECT * FROM emp WHERE comm IS NULL;
```

- LIKE

A n'utiliser que si % et/ou _ (**sinon utiliser =**)

```
SELECT ename FROM emp WHERE ename LIKE '_AR%';
```

- DISTINCT

```
SELECT DISTINCT job FROM emp;
```

Et non :

```
SELECT job FROM emp  
GROUP BY job;
```

- IN

```
SELECT ename, deptno FROM emp WHERE deptno IN (10,20);
```

- **ECRITURE NON VALIDE (Pas de SELECT dans le FROM!) :**

```
SELECT *  
FROM (SELECT * FROM EMP)  
WHERE comm IS NULL;
```

Jointures

- Jointure interne : **INNER JOIN** ou **JOIN** (syntaxe SQL2)

```
SELECT ename, dname  
FROM emp
```

```
JOIN dept ON emp.deptno=dept.deptno;
```

-> Affiche les employés ayant un service (et uniquement ceux-là)

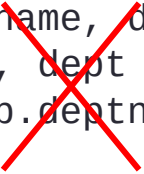
OU en utilisant des alias :

```
SELECT e.ename, d.dname  
FROM emp e
```

```
JOIN dept d ON e.deptno=d.deptno;
```

- Utiliser de préférence la syntaxe SQL2 à la suivante (syntaxe SQL1) :

```
SELECT ename, dname  
FROM emp, dept  
WHERE emp.deptno=dept.deptno;
```



Jointures

- Jointure externe :

- A gauche : LEFT OUTER JOIN ou LEFT JOIN

- SELECT ename, dname
FROM emp

- LEFT JOIN dept ON emp.deptno=dept.deptno;

- > Affiche tous les employés y compris ceux qui n'ont pas de service

- A droite : RIGHT OUTER JOIN ou RIGHT JOIN

- SELECT ename, dname
FROM emp

- RIGHT JOIN dept ON emp.deptno=dept.deptno;

- > Affiche tous les départements y compris ceux qui n'ont pas d'employé

Fonctions d'agrégation

- Utilisation de COUNT, SUM, AVG, MAX, MIN, etc.

```
SELECT AVG(sal) "Moyenne"
FROM emp
```

```
SELECT deptno, dname, AVG(sal) "Moyenne"
FROM emp
      JOIN dept ON emp.deptno=dept.deptno;
GROUP BY deptno, dname
```

Tous les champs du
SELECT hormis celui sur
lequel porte la fonction
d'agrégat doivent être
dans la clause GROUP BY

```
SELECT deptno, COUNT(*)
FROM emp
WHERE job <> 'SALESMAN'
GROUP BY deptno
HAVING COUNT(*) > 3;
```

Pas de restriction sur des fonctions
d'agrégat dans le WHERE (que des
restrictions simples)

Les restrictions sur les fonctions d'agrégat
(et uniquement celles-ci) sont dans la
clause HAVING

Fonctions d'agrégation

- NE PAS UTILISER DE TABLE DERIVEE SUR DES REQUETES SIMPLES

```
SELECT    deptno, COUNT(*)  
FROM      emp  
GROUP BY  deptno  
HAVING    COUNT(*) > 3;
```

```
SELECT    deptno, COUNT(*)  
FROM      emp  
WHERE      (SELECT COUNT(DEPTNO) FROM emp) > 3  
GROUP BY  deptno
```



-- Le Coût de cette requête est le double de la précédente !

Sous-interrogations simples (non synchronisées)

- Exemple : Employés travaillant à DALLAS

```
SELECT  e.ename, e.sal, e.comm
FROM    emp e
JOIN    dept d ON e.deptno = d.deptno
WHERE   d.loc = 'DALLAS';
```

La sous-interrogation est indépendante de l'interrogation principale

```
SELECT  e.ename, e.sal, e.comm
FROM    emp e
WHERE   e.deptno =
        (SELECT  d.deptno FROM dept d
         WHERE   d.loc = 'DALLAS');
```

1 attribut de même type !

- Exemple : Employés travaillant à DALLAS et dont le salaire est > 2000

```
SELECT  e.ename, e.deptno
FROM    emp e
WHERE   e.empno IN
        (SELECT  e1.empno
         FROM    emp e1
         WHERE   e1.deptno IN
                 (SELECT  d.deptno
                  FROM    dept d
                  WHERE   d.loc = 'DALLAS')
         OR      e1.sal > 2000)
```

Sous-interrogations simples (non synchronisées)

- Exemple : Employés habitant au même endroit que leur lieu de travail

La sous-interrogation est indépendante de l'interrogation principale

```
SELECT  e.ename, e.adr
FROM    emp e
        JOIN dept d ON e.deptno = d.deptno AND e.adr=d.loc;
```

```
SELECT  e.ename, e.adr
FROM    emp e
WHERE   (e.deptno, e.adr) IN (
```

2 attributs de même type ! On peut avoir N attributs de la même façon

```
        SELECT deptno, loc
        FROM dept
        );
```

Sous-interrogations

- Requête imbriquée non synchronisée :
 - Sous-requête est indépendante de la requête principale :
 1. Le sous-requête s'exécute et renvoie un résultat
 2. Ce résultat est utilisé lors de l'exécution de la requête principale à des fins de restriction
- Requête imbriquée synchronisée :
 - = Sous-interrogation synchronisée
 - Manipule des colonnes d'une table du niveau supérieur
 - Est exécutée une fois pour chaque enregistrement extrait par la requête de niveau supérieur.
 - Peut également être utilisée dans les ordres UPDATE et DELETE
 - Il est préférable d'utiliser des alias de tables pour pouvoir mieux manipuler les colonnes de tables de différents niveaux
 - La sous-interrogation peut ramener 1 ou plusieurs lignes
 - Opérateurs possibles : =, >, <, >=, <=, IN, NOT IN, EXISTS, NOT EXISTS
 - Syntaxe :

```

SELECT c
FROM Table1 alias1
WHERE colonne(s) opérateur (SELECT d
                             FROM Table2 alias2
                             WHERE alias1.x opérateur alias2.y)
  
```

Référence

Requête imbriquée synchronisée

- Afficher les employés dont le salaire est supérieur au salaire moyen des employés de leur département

```
SELECT ename
```

```
FROM emp e1
```

```
WHERE sal > (
```

```
    SELECT avg(sal)
```

```
    FROM emp e2
```

```
    WHERE e1.deptno=e2.deptno
```

```
)
```

- Pour chaque ligne de la table e1 (emp), c'est-à-dire pour chaque employé, la sous-requête va être exécutée et calculer la moyenne des salaires des employés appartenant au même département que celui de l'employé en cours de la table e1.

Requête imbriquée synchronisée

1^{er} enregistrement de e1 :

e1 (emp)

empno	ename	sal	deptno
1	TOTO	2000	20

JOINTURE

Sous-requête (e2) : calcul de la moy.
des salaires des employés du dept
20 (dept de e1)

Avg(sal)	deptno
1560	20



ename
TOTO

2^{ème} enregistrement de e1, etc.

e1 (emp)

empno	ename	sal	deptno
2	TITI	1000	30

JOINTURE

Sous-requête (e2) : calcul de la moy.
des salaires des employés du dept
30 (dept de e1)

Avg(sal)	deptno
2300	30



RIEN

Requête imbriquée synchronisée

- Afficher les employés seuls dans leur service :

```
SELECT    e1.empno, e1.ename
FROM      emp e1
WHERE     e1.deptno NOT IN
          (SELECT e2.deptno
           FROM emp e2
           WHERE e2.empno != e1.empno)
```

La sous-requête renvoie tous les départements sauf celui de l'employé sur lequel on est positionné dans e1.

S'il y a pas d'autres employés que e1 dans son département, le département de e1 n'est pas renvoyé par la sous-requête => e1 est seul.

Sinon, le département de e1 est renvoyé et il n'est pas seul.

Requête imbriquée synchronisée : EXISTS / NOT EXISTS

- Afficher les employés seuls dans leur service :

```
SELECT      e1.empno, e1.ename
FROM        emp e1
WHERE       NOT EXISTS
            (SELECT e2.empno
             FROM   emp e2
             WHERE  e2.deptno = e1.deptno
             AND    e2.empno != e1.empno)
```


Requête imbriquée synchronisée : EXISTS / NOT EXISTS

- EXISTS et NOT EXISTS nécessitent toujours l'utilisation d'une sous-interrogation synchronisée
 - Ce n'est pas le cas de IN ou NOT IN par exemple qui peuvent être utilisées dans les sous-interrogations non synchronisées
- **ATTENTION, contrairement aux autres opérateurs, il n'y a pas de champ devant EXISTS ou NOT EXISTS.**
 - Il s'agit juste de déterminer si la sous-requête renvoie ou pas des lignes
- EXISTS :
 - Permet d'interrompre la sous-interrogation dès le 1^{er} enregistrement trouvé
 - FALSE est retourné si aucun enregistrement n'est extrait par la sous-requête
- NOT EXISTS :
 - TRUE est retourné si aucun enregistrement n'est extrait par la sous-requête
 - Peut être utilisé pour écrire des jointures externes à la place de LEFT JOIN ou RIGHT JOIN.

Requête imbriquée synchronisée : EXISTS / NOT EXISTS

- Comme il s'agit juste de savoir si des lignes sont renvoyées ou pas par la sous-requête (et non ce qu'il y a dedans) :

```
SELECT      e1.empno, e1.ename
FROM        emp e1
WHERE       NOT EXISTS
            (SELECT e2.empno
             FROM   emp e2
             WHERE  e2.deptno = e1.deptno
             AND    e2.empno != e1.empno);
```

- La requête précédente peut s'écrire :

```
SELECT      e1.empno, e1.ename
FROM        emp e1
WHERE       NOT EXISTS
            (SELECT 'X'
             FROM   emp e2
             WHERE  e2.deptno = e1.deptno
             AND    e2.empno != e1.empno);
```

Division

- La division ($\div$) est un opérateur algébrique
- Permet de diviser une relation (ou partie de) par une autre relation (ou partie de)
- A utiliser quand on souhaite comparer un ensemble avec un groupe de référence
- Permet de traduire le terme « tous les » (placé après le verbe conjugué dans la proposition subordonnée) des requêtes

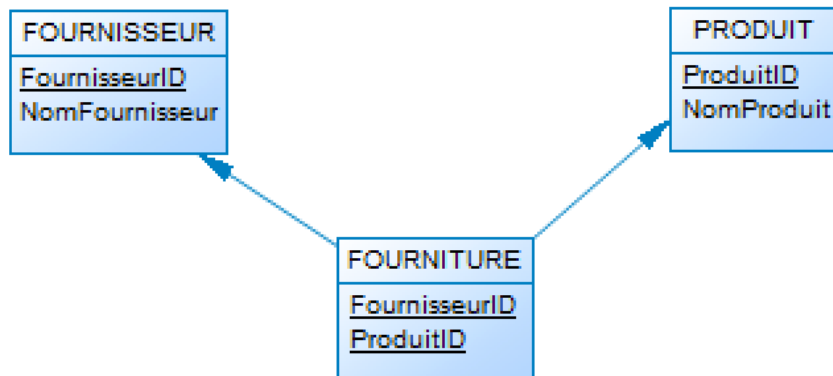
T1		T2	
Dupont	INFO	INFO	
Dupont	GEII	GEII	
Dupont	R&T	R&T	
Durant	R&T		
Martin	INFO		
Martin	GEII		
		R	Dupont

Quels sont les enregistrements de T1 qui sont associés à tous les enregistrements de T2 ?

Réponse : $R = T1 \div T2$

Division : NOT EXISTS

- En SQL, l'opérateur de division n'a pas été implanté.
- La solution la plus performante consiste à écrire plusieurs requêtes NOT EXISTS synchronisées
- Exemple : « Afficher le nom des fournisseurs qui fournissent **tous** les produits »



Fournisseurs qui
fournissent des produits
(*Fournisseur JOIN Fourniture*)

Fournisseur1	P1
Fournisseur1	P2
Fournisseur2	P1
Fournisseur2	P2
Fournisseur2	P3
Fournisseur3	P3

Produit

P1

P2

P3

R

Fournisseur2

Division : NOT EXISTS

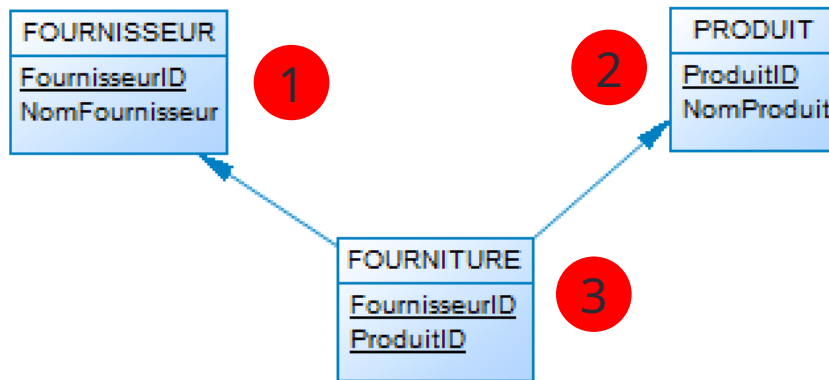
- Afficher le nom des fournisseurs qui fournissent **tous** les produits
 - ⇔ Noms des fournisseurs pour lesquels il n'existe pas de produits non fournis
 - ⇔ Noms des fournisseurs pour lesquels il n'existe pas de produits pour lesquels il n'existe pas de fourniture (de ces produits par ces fournisseurs)

```

SELECT NomFournisseur
FROM Fournisseur F
WHERE NOT EXISTS
  (SELECT 'X'
   FROM Produit P
   WHERE NOT EXISTS
     (SELECT 'X'
      FROM Fourniture Ft
      WHERE Ft.FournisseurID = F.FournisseurID
            AND Ft.ProduitID = P.ProduitID)
  )

```

Division : NOT EXISTS



```

SELECT NomFournisseur
FROM Fournisseur F
WHERE NOT EXISTS
  (SELECT 'X'
   FROM Produit P
   WHERE NOT EXISTS
     (SELECT 'X'
      FROM Fourniture Ft
      WHERE Ft.FournisseurID = F.FournisseurID
            AND Ft.ProduitID = P.ProduitID)
  )
  
```

Quand il y a 3 tables, la façon de procéder est simple :

- En position 1 : la table dont on souhaite afficher les données
- En position 3 : la table qui fera la jointure avec les 2 autres
- En position 2 : le groupe de référence

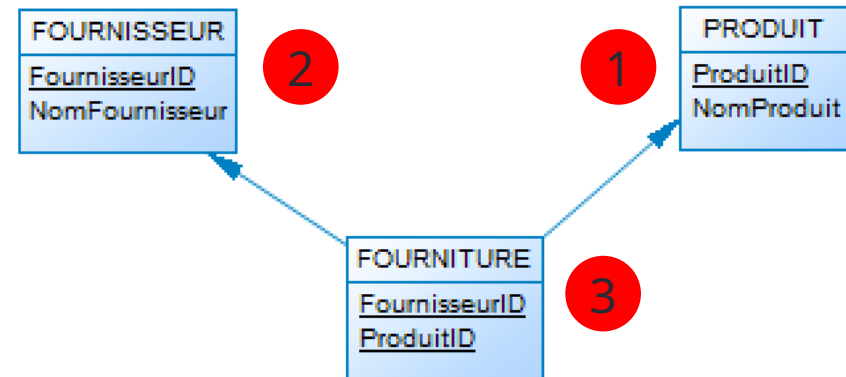
Quand il y a plus de 3 tables, c'est plus compliqué : il faut réfléchir pour réduire à 3 relations (regrouper 2 tables par une jointure -> 1 nouvelle relation)

Division : NOT EXISTS

- Afficher le nom des produits qui sont fournis par **tous** les fournisseurs
 - ⇔ Noms des produits pour lesquels il n'existe pas de fournisseurs qui ne fournissent pas ces produits
 - ⇔ Noms des produits pour lesquels il n'existe pas de fournisseurs pour lesquels il n'existe pas de fourniture (de ces produits par ces fournisseurs)

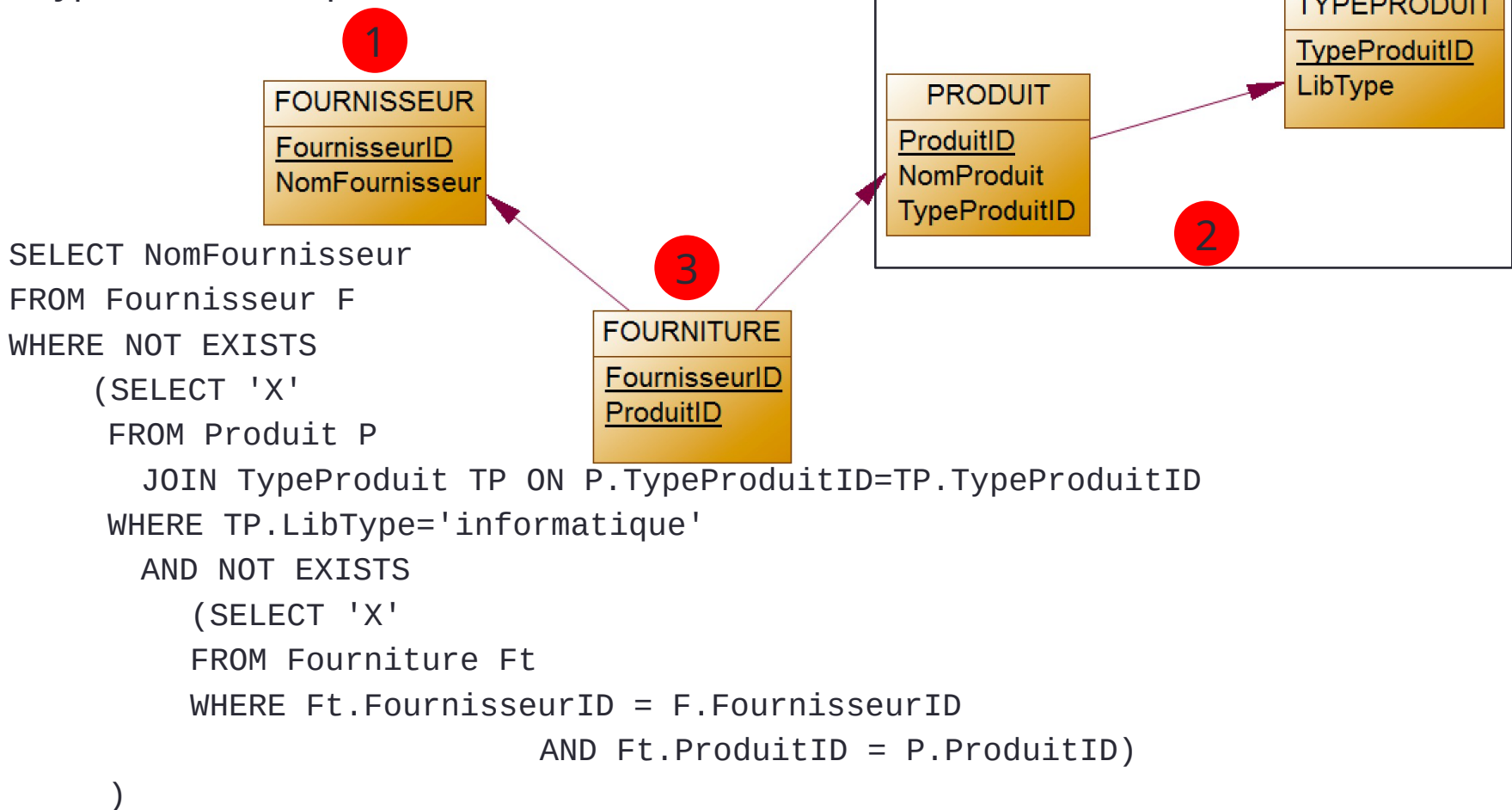
```

SELECT NomProduit
FROM Produit P
WHERE NOT EXISTS
  (SELECT 'X'
   FROM Fournisseur F
   WHERE NOT EXISTS
     (SELECT 'X'
      FROM Fourniture Ft
      WHERE Ft.FournisseurID = F.FournisseurID
            AND Ft.ProduitID = P.ProduitID))
  
```



Division : NOT EXISTS

- Exemple : Afficher le nom des fournisseurs qui fournissent **tous** les produits de type « informatique »



Division : COUNT

- Possible d'écrire des divisions en utilisant des fonctions d'agrégation COUNT
- Moins performant que d'utiliser NOT EXISTS notamment quand le nombre de lignes des tables est très élevé (temps de calcul longs).
- Exemple : « Afficher le nom des fournisseurs qui fournissent **tous** les produits »
 - Une sous-requête calcule le nombre de produits contenus dans la table PRODUIT
 - La requête principale calcule le nombre de produits **différents** fournis par fournisseur et le compare au résultat de la sous-requête

```
SELECT NomFournisseur
FROM Fournisseur F
  JOIN Fourniture Ft ON F.FournisseurID=Ft.FournisseurID
GROUP By NomFournisseur
HAVING count (DISTINCT produitID)=(
                                SELECT count(*) FROM produit
                                );
```

INSERT
UPDATE
DELETE

INSERT

- Exemple sans nom des colonnes

```
INSERT INTO dept  
VALUES (50, null, null)
```

- Exemple avec noms de colonnes

```
INSERT INTO dept (deptno)  
VALUES (50)
```

- => Pour une meilleure évolutivité, il est préférable de nommer les colonnes utilisées

- Exemples insert avec select (permet d'insérer plusieurs lignes) :

```
INSERT INTO essai  
SELECT *  
FROM emp;
```

```
INSERT INTO emp (empno, ename, job, sal, deptno)  
SELECT 100, 'Dupont Jean', 'Vendeur', 2000, deptno FROM DEPT  
WHERE LOC='Paris'
```

UPDATE

- Exemples :
 - sans WHERE : augmentation des salaires de 5 %

```
UPDATE emp
SET      sal = sal * 1.05;
```
 - avec WHERE : augmentation du salaire des vendeurs

```
UPDATE emp
SET      sal = sal * 1.05
WHERE    job = 'SALESMAN';
```
- Exemple : sous-requête synchronisée

```
UPDATE emp e
SET      sal = (SELECT AVG(e2.sal)
                FROM      emp e2
                WHERE     e.deptno = e2.deptno);
```

DELETE

- Exemples :
 - sans WHERE : suppression de tous les services
DELETE FROM dept;
 - avec WHERE : suppression des employés ayant une commission = 0
DELETE FROM emp WHERE comm = 0
- Exemple avec WHERE et SELECT : suppression des employés travaillant à DALLAS

```
DELETE FROM emp
WHERE deptno =
      (SELECT deptno
       FROM dept
       WHERE loc = 'DALLAS')
```